

iOS Prompt Vocabulary

Common nouns and verbs for asking Codex to build SwiftUI-first iPhone and iPad apps.

Use these words to tell Codex what surface, behavior, framework, state, test, or release step you mean. The goal is precision, not memorizing Apple APIs.

Fast Prompt Recipe

- Name the platform and UI framework: SwiftUI iOS app, UIKit only if the existing app already uses it.
- Name the navigation shape: tab bar, navigation stack, split view, sheet, popover, alert, or confirmation dialog.
- Name the state and storage: state, binding, observable model, SwiftData, Core Data, CloudKit, Keychain, or UserDefaults.
- Name the system integration: App Intents, widgets, notifications, share sheet, StoreKit, background tasks, or deep links.
- Name verification and release work: preview, simulator, unit test, UI test, archive, TestFlight, App Review, and App Store Connect.

Fill In These Slots

Slot	Example wording
App type	Build a SwiftUI iOS app for iPhone and iPad...
Screens	Create Today, Projects, Detail, and Settings screens...
Navigation	Use a tab bar with each tab wrapped in a NavigationStack...
Data model	Persist TaskItem records with SwiftData and expose observable view models...
Interaction	Support search, filters, swipe actions, pull to refresh, edit sheets, and undo where appropriate...
Quality bar	Add accessibility labels, Dynamic Type support, unit tests, UI tests, and run on the iPhone 17 Pro simulator...
Release path	Prepare the app for TestFlight: set bundle identifier, signing, app icon, build number, archive, and upload...

Project And Xcode Nouns

Use these when the work is about project setup, build configuration, assets, signing, or capabilities.

Term	Means	Useful prompt wording
Xcode project	The .xcodeproj that contains the app target, files, build settings, and schemes.	Create a new Xcode project named FocusDeck.
workspace	A container that can hold multiple projects and Swift packages.	Use a workspace only if the repo already has one.
target	The buildable app, extension, widget, or test bundle.	Add a widget extension target and share models with the app target.
scheme	The named Xcode build/run/test/profile configuration.	Add Makefile targets that build the FocusDeck scheme.
bundle identifier	The unique reverse-DNS app id used for signing and App Store Connect.	Use bundle identifier com.cseibert.FocusDeck.
build number	The monotonically increasing internal build string.	Increment the build number before archiving.
version	The user-visible marketing version, like 1.0.	Set the marketing version to 1.0 for the first TestFlight build.
asset catalog	The Xcode catalog for app icons, colors, images, and symbols.	Add named colors and app icons to Assets.xcassets.
app icon	The icon users see on the Home Screen, App Store, Spotlight, and Settings.	Create a simple 1024 by 1024 app icon and add it to the asset catalog.
launch screen	The placeholder screen iOS shows while the app starts.	Configure a simple launch screen that matches the app background color.
entitlement	A signing permission for protected capabilities such as iCloud, Keychain, or Push Notifications.	Add the iCloud entitlement only if CloudKit sync is implemented.
capability	An Xcode Signing and Capabilities feature that configures services and entitlements.	Enable Push Notifications and Background Modes capabilities.
provisioning profile	The signing profile that ties app id, certificates, entitlements, and devices together.	Use automatic signing and document any required provisioning profile.
Info.plist	Metadata and platform declarations packaged with the app.	Add camera and photo library usage descriptions to Info.plist.
Swift package	A dependency managed by Swift Package Manager.	Add the dependency as a Swift package and pin the version.
preview	A SwiftUI canvas preview for seeing a view without launching the full app.	Add previews with sample data for the detail screen.

UI Surface And Navigation Nouns

Use these to describe where people go, what appears on top, and what shape the app takes on iPhone and iPad.

Term	Means	Useful prompt wording
app	The installed iOS application as a whole.	Build the app as a native SwiftUI iOS app.
scene	A SwiftUI app entry point or window-like lifecycle unit.	Use a WindowGroup scene for the main app UI.
view	A SwiftUI reusable UI component.	Extract the task row into a TaskRow view.
screen	A user-facing full page or destination.	Create a ProjectDetail screen.
NavigationStack	A stack-based navigation container for pushing detail screens.	Wrap the list in a NavigationStack and push ProjectDetail from each row.
NavigationSplitView	A two- or three-column navigation layout, especially useful on iPad.	Use NavigationSplitView on iPad with sidebar, content, and detail columns.
tab bar	Bottom navigation for switching between major areas.	Use a tab bar with Today, Projects, Search, and Settings tabs.
toolbar	A row of actions in the navigation bar or bottom bar.	Add toolbar buttons for add, edit, and filter.
sheet	A modal surface for a focused task that can be dismissed.	Present the add-task form as a sheet.
full-screen cover	A modal presentation that takes the whole screen.	Use a full-screen cover for onboarding.
popover	A small transient panel, more common on iPad.	Show filter options in a popover on iPad and a sheet on iPhone.
alert	A blocking message that asks for acknowledgment.	Show an alert when saving fails.
confirmation dialog	A list of choices for a destructive or consequential action.	Use a confirmation dialog before deleting a project.
context menu	Actions revealed by long-pressing or secondary-clicking an item.	Add a context menu with Rename, Duplicate, and Delete.
sidebar	A leading navigation column, mostly for iPad layouts.	Use a sidebar for project groups on regular-width iPad.
deep link	A URL or universal link that opens a specific destination.	Support deep links to open a task detail screen.

Layout, Controls, And Visual Nouns

Use these to describe the visible pieces of a screen and how they should adapt across device sizes.

Term	Means	Useful prompt wording
List	A scrolling collection of rows, often with sections and swipe actions.	Use a List with sections for overdue, today, and later tasks.
Form	A grouped input surface for settings or editing data.	Build Settings as a Form with sections.
Section	A labeled group inside a list or form.	Group notification settings into a Section.
ScrollView	A scrollable freeform layout.	Use ScrollView for the dashboard content.
VStack/HStack/ZStack	Vertical, horizontal, and layered SwiftUI layout containers.	Use VStack for the form fields and HStack for row metadata.
Grid/LazyVGrid	Grid layout for cards or thumbnails.	Use LazyVGrid for the template picker.
safe area	The visible region that avoids notches, home indicator, and system bars.	Respect the safe area and avoid placing controls behind the home indicator.
size class	Compact or regular layout category used to adapt between iPhone and iPad.	Use size classes to switch from tabs on iPhone to split view on iPad.
Dynamic Type	System text scaling for accessibility.	Support Dynamic Type without clipped text.
SF Symbols	Apple's symbol icon set.	Use SF Symbols for toolbar and tab icons.
system color	Adaptive colors that work in light and dark mode.	Use system background, secondary background, accent, and semantic colors.
material	A system blur/translucency treatment.	Use platform materials sparingly for overlays.
Liquid Glass	Apple's current system visual language for native controls and materials.	Prefer native SwiftUI components so Liquid Glass behavior comes from the system.
Button	A tappable command.	Use prominent buttons only for primary actions.
Toggle	A binary on/off control.	Use a Toggle for notification preferences.
Picker	A control for choosing one value from known options.	Use a segmented Picker for list, board, and calendar modes.
TextField	A single-line text input.	Use TextField for task title and validate empty titles.
Search field	A searchable modifier or search UI.	Add searchable text to the project list.
Swipe action	Row actions revealed by swiping.	Add swipe actions for complete and delete.

Data, State, And Architecture Nouns

Use these when asking Codex to store data, load remote content, or keep views in sync.

Term	Means	Useful prompt wording
model	A type that represents app data.	Create a TaskItem model with title, notes, due date, and status.
view model	An object that prepares state and actions for views.	Add a ProjectListViewModel if the view logic becomes complex.
state	Local view-owned mutable data.	Use @State for transient form fields.
binding	A two-way connection between a parent value and child control.	Pass a Binding into the edit sheet.
observable model	A model object that tells SwiftUI when data changes.	Make the task store observable so lists update automatically.
environment	Values injected into a view tree.	Use environment for model context and app-wide settings.
async task	Concurrent work launched with Swift concurrency.	Load remote suggestions with an async task and show loading and error states.
actor	A concurrency-safe object for shared mutable state.	Use an actor for the sync engine if it mutates shared queues.
service	A boundary around external behavior such as networking or notifications.	Extract notification scheduling into a NotificationService.
repository	A data access layer that hides persistence details.	Use a repository so tests can inject an in-memory store.
SwiftData	Apple's modern Swift persistence framework.	Persist tasks with SwiftData and add migration notes.
Core Data	Older but powerful object graph persistence framework.	Use Core Data only if the existing app already uses it.
CloudKit	Apple's iCloud sync backend.	Sync projects with CloudKit and handle offline edits.
UserDefaults	Small preferences storage.	Store onboarding completion in UserDefaults.
Keychain	Secure storage for secrets.	Store auth tokens in Keychain, not UserDefaults.
cache	Temporary local copy to improve speed or offline behavior.	Cache remote avatars and expire them after 24 hours.

Platform Feature, Testing, And Release Nouns

Use these when the app integrates with iOS or when you want Codex to prove the app works.

Term	Means	Useful prompt wording
App Intents	System actions that expose app features to Shortcuts, Siri, Spotlight, and widgets.	Add App Intents for creating and completing tasks.
widget	A Home Screen or Lock Screen glance surface.	Add a widget extension that shows today's top tasks.
Live Activity	A glanceable ongoing activity on the Lock Screen and Dynamic Island.	Use a Live Activity for an active timer.
notification	A local or push message delivered by the system.	Schedule local notifications for due tasks.
background task	System-managed background work.	Use a background task to refresh sync state.
share sheet	System UI for sharing content to other apps.	Add a share sheet for exporting a project summary.
permission prompt	A system request for access to protected data or sensors.	Request photo access only when the user taps Import.
StoreKit	Apple framework for in-app purchases and subscriptions.	Use StoreKit for a Pro subscription and include sandbox tests.
simulator	A virtual iPhone or iPad for building and running locally.	Run on the iPhone 17 Pro simulator after implementation.
unit test	A focused code-level test.	Add unit tests for task recurrence logic.
UI test	An end-to-end XCTest that drives the interface.	Add a UI test for creating and completing a task.
snapshot test	A rendered UI comparison test, usually project-specific.	Add snapshot tests only if the repo already uses a snapshot framework.
Instruments	Apple profiling tools for performance, memory, energy, and hangs.	Profile scrolling with Instruments if the list stutters.
crash log	A report from a terminated app.	Symbolicate the crash log and identify the failing stack frame.
App Store Connect	Apple's portal for builds, TestFlight, metadata, review, pricing, and release.	Create the App Store Connect checklist for this app.

Verbs That Help Codex Act

Use verbs that name the engineering move. They reduce ambiguity more than extra adjectives do.

Verb	Means	Useful prompt wording
scaffold	Create the project or feature skeleton.	Scaffold a SwiftUI iOS app with app, tests, and Makefile targets.
implement	Build real behavior, not just placeholders.	Implement the create-task flow end to end.
extract	Move repeated UI or logic into a reusable type.	Extract the repeated row layout into TaskRow.
wire	Connect UI, state, services, and navigation.	Wire the Add button to present the edit sheet and save through SwiftData.
bind	Connect controls to mutable state.	Bind the title TextField to the draft task title.
present	Show a modal or transient surface.	Present the filter picker as a sheet on iPhone.
dismiss	Close a modal or destination.	Dismiss the edit sheet after a successful save.
push	Navigate to a deeper screen in a stack.	Push ProjectDetail when a project row is tapped.
validate	Check input and show useful errors.	Validate empty titles before saving.
persist	Save data locally.	Persist tasks with SwiftData.
fetch	Load data from storage or network.	Fetch projects sorted by updated date.
sync	Reconcile local and remote data.	Sync changes to CloudKit and resolve conflicts by latest edit timestamp.
cache	Store temporary copies for speed or offline use.	Cache downloaded thumbnails.
debounce	Wait for typing to pause before acting.	Debounce search input before filtering a large list.
request permission	Ask for protected system access.	Request notification permission during onboarding.
mock	Replace a real service in tests or previews.	Mock the sync service in SwiftUI previews and unit tests.
profile	Measure runtime performance.	Profile launch time and list scrolling.
localize	Prepare user-facing strings for translation.	Localize visible strings and use String Catalogs.
archive	Create a distributable build in Xcode.	Archive the release build for App Store Connect.
upload	Send a build to App Store Connect.	Upload the archive for TestFlight.

Ambiguous Words To Replace

These are common non-engineer words that can make an agent guess. Replace them with the platform terms on the right.

Instead of	Probably means	Say this instead
page	screen, view, tab, or destination	Create a Settings screen and implement it as a SwiftUI view.
popup	sheet, popover, alert, menu, or confirmation dialog	Use a confirmation dialog for destructive choices.
menu	toolbar menu, context menu, or tab bar	Add a context menu to each task row.
database	SwiftData, Core Data, CloudKit, SQLite, or backend API	Use SwiftData for local persistence and CloudKit for sync.
login stuff	authentication flow, credentials, Keychain, session refresh	Store tokens in Keychain and refresh the session on launch.
make it native	SwiftUI controls, HIG layout, accessibility, system colors	Use native SwiftUI controls and system colors.
save button	persist draft, validate, dismiss, show error	Validate the form, persist the task, then dismiss the sheet.
background refresh	BackgroundTasks, push, silent push, or sync on launch	Use BackgroundTasks for periodic sync and also sync on launch.
publish it	TestFlight, App Review, App Store release, or ad hoc install	Prepare a TestFlight build and list App Review metadata still needed.
phone app	iOS app for iPhone, or universal iPhone/iPad app	Build a universal SwiftUI iOS app that adapts for iPhone and iPad.

Release And Publishing Words

Term or step	Prompt wording
Prepare for distribution	Check bundle identifier, app icon, launch screen, version, build number, signing, entitlements, and required privacy strings.
Archive	Create an Xcode archive for Release configuration.
Upload build	Upload the archive to App Store Connect.
TestFlight	Distribute beta builds, add internal or external testers, collect feedback, and iterate.
Export compliance	Answer App Store Connect encryption questions before beta or App Review submission when required.
App privacy	Prepare privacy labels and data collection notes for App Store Connect.
App Review	Submit an app version and required metadata for Apple's review process.
Phased release	Release gradually after approval instead of to all users at once.
Crash and metrics	Review TestFlight or App Store Connect sessions, crashes, and feedback before wider release.

Example Prompts

New iOS app

Build a SwiftUI iOS app named FocusDeck for iPhone and iPad. Use a tab bar with Today, Projects, and Settings. Wrap each tab in a UINavigationController. Persist TaskItem and Project models with SwiftData. Add searchable lists, swipe actions for complete/delete, an edit sheet, Dynamic Type, accessibility labels, unit tests, and a UI test for creating a task. Run it on the iPhone 17 Pro simulator.

Feature addition

Add App Intents for creating a task and marking one complete. Keep the implementation shared with the existing task service. Add shortcuts-friendly titles, parameter validation, and unit tests for the intent handlers.

Release prep

Prepare the app for TestFlight. Verify bundle identifier, signing, app icon, launch screen, build number, privacy usage descriptions, and export compliance notes. Archive a Release build and list any App Store Connect metadata I still need to provide.

Reference Anchors

Source	URL
Apple SwiftUI documentation	https://developer.apple.com/documentation/swiftui
Apple UIKit documentation	https://developer.apple.com/documentation/uikit
Apple Human Interface Guidelines	https://developer.apple.com/design/human-interface-guidelines
Xcode app distribution	https://developer.apple.com/documentation/xcode/preparing-your-app-for-distribution/
App Store Connect TestFlight overview	https://developer.apple.com/help/app-store-connect/test-a-beta-version/testflight-overview/
App Store Connect App Review overview	https://developer.apple.com/help/app-store-connect/manage-submissions-to-app-review/overview-of-submitting-for-review/